EXPRESS.JS

SUNITA D RATHOD

EXPRESS JS

Express is a fast, assertive, essential and moderate web framework of Node.js.

You can assume express as a layer built on the top of the Node.js that helps manage a server and routes.

It provides a robust set of features to develop web and mobile applications.

Express is an open-source that is developed and maintained by the Node.js foundation.

Express.js

With Express, you can easily handle routes, requests, and responses, which makes the process of creating robust and scalable applications much easier.

Moreover, it is a lightweight and flexible framework that is easy to learn and comes loaded with middleware options.

Express was developed by **TJ Holowaychuk** and is maintained by the Node.js foundation and numerous open source contributors.

Express.js provides all the features of a modern web framework, such as templating, static file handling, connectivity with SQL and NoSQL databases.

Features of Express.js

- **Middleware and Routing**: Define clear pathways (routes) within your application to handle incoming HTTP requests (GET, POST, PUT, DELETE) with ease. Implement reusable functions (middleware) to intercept requests and create responses, adding functionalities like authentication, logging, and data parsing.

Features of Express.js

- **Minimalistic Design**: Express.js follows a simple and minimalistic design philosophy. This simplicity allows you to quickly set up a server, define routes, and handle HTTP requests efficiently. It’s an excellent choice for building web applications without unnecessary complexity.
- **Middleware :** Middleware is a request handler that has access to the application's request-response cycle.
- **Routing :**It refers to how an application's endpoint's URLs respond to client requests.

Features of Express.js

- **Flexibility and Customization**: Express.js doesn't impose a strict application architecture. You can structure your code according to your preferences. Whether you're building a RESTful API or a full-fledged web app, Express.js adapts to your needs.
- **Templating Power:** Incorporate templating engines like Jade or EJS to generate dynamic HTML content, enhancing user experience.

Features of Express.js

- **Static File Serving:** Effortlessly serve static files like images, CSS, and JavaScript from a designated directory within your application.
- **Node.js Integration:** Express.js seamlessly integrates with the core functionalities of Node.js, allowing you to harness the power of asynchronous programming and event-driven architecture.

Advantages of Using Express With Node.js

- Express is Un opinionated, and we can customize it.
- For request handling, we can use Middleware.
- A single language is used for frontend and backend development.
- Express is fast to link it with databases like MySQL, MongoDB, etc.
- Express allows dynamic rendering of HTML Pages based on passing arguments to templates.

Commands for installation

1) Node - - version

2) npm - - version

3) Start your terminal/cmd, create a new folder named hello-world and cd (create directory) into it

mkdir hello-world

cd hello-world

4) Now to create the package.json file using npm , use the following code.

npm init

Commands for installation

5)Now we have our package.json file set up, we will further install Express. To install Express and add it to our package.json file, use the following command –

npm install --save express

6)

dir node_modules

Commands for installation

This is all we need to start development using the Express framework.

• To make our development process a lot easier, we will install a tool from npm, nodemon.

• This tool restarts our server as soon as we make a change in any of our files, otherwise

we need to restart the server manually after each file modification.

To install nodemon, use the following command

–npm install -g nodemon

Getting started with express

Import the ‘**express**’ module to create a web application using Node.js.

Initialize an Express app using **`const app = express();`**.

Add **routes (endpoints)** and **middleware** functions to handle requests and perform tasks like authentication or logging.

Specify a port **(defaulting to 3000)** for the server to listen on.

ExpressJS - Hello World

Create a new file called **index.js** and type the following in it.

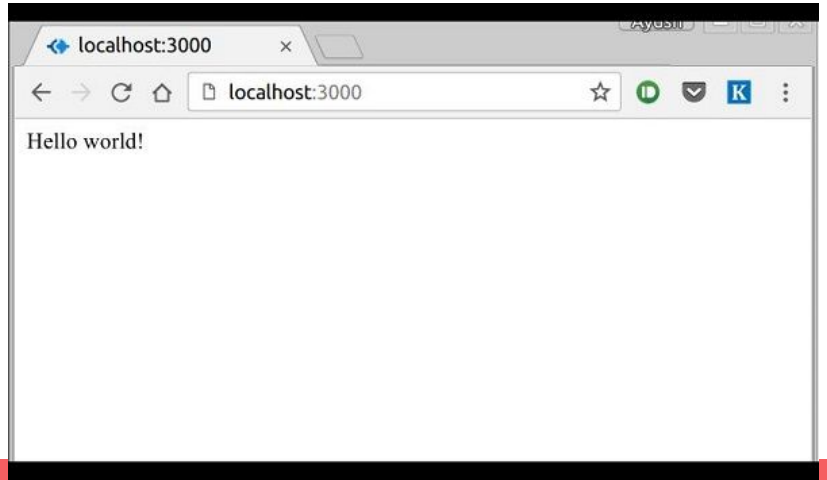
```
var express = require('express');

var app = express();

app.get('/', function(req, res){

  res.send("Hello world!");

});

 app.listen(3000);
```

ExpressJS - Hello World

Save the file, go to your terminal and type the following.

```
nodemon index.js
```

This will start the server. To test this app, open your browser and go to http://localhost:3000 and a message will be displayed as in the following screenshot.



ExpressJS - Routing

The following function is used to define routes in an Express application −

app.method(path, handler)

This METHOD can be applied to any one of the HTTP verbs – get, set, put, delete. An alternate method also exists, which executes independent of the request type.

Path is the route at which the request will run.

Handler is a callback function that executes when a matching request type is found on the relevant route.

app.get(route, callback)

This function tells what to do when a get request at the given route is called.

• The callback function has 2 parameters, request(req) and response(res).

• The request object(req) represents the HTTP request and has properties for the request query string, parameters, body, HTTP headers, etc.

• Similarly, the response object represents the HTTP response that the Express app sends when it receives an HTTP request.

res.send()

This function takes an object as input and it sends this to the requesting client.

- Here we are sending the string "Hello World!".

app.listen(port, [host], [backlog],[callback])

port	A port number on which the server should accept incoming requests
host	Name of the domain. You need to set it when you deploy your apps to the cloud.
backlog	The maximum number of queued pending connections. The default is 511.
callback	An asynchronous function that is called when the server starts listening for requests.

ExpressJS - Routing

We can also have multiple different methods at the same route.

```
var express = require('express');

var app = express();

app.get('/hello', function(req, res){

   res.send("Hello World!");

});

app.post('/hello', function(req, res){

   res.send("You just called the post method at '/hello'!\n");

});

app.listen(3000);
```

To test this request, open up your terminal and use cURL to execute the following request –

```
curl -X POST

"http://localhost:3000/hello"
```

```
ayushgp@swaggy:~/hello-world$ curl -X POST "http://localhost:3000/hello"
You just called the post method at '/hello'!
ayushgp@swaggy:~/hello-world$
```

ExpressJS - Routing

A special method, **all**, is provided by Express to handle all types of http methods at a particular route using the same function. To use this method, try the following.

```
app.all('/test', function(req, res){

   res.send("HTTP method doesn't have any effect on this route!");

});
```

This method is generally used for defining middleware

Express express.Router() Function

The express.Router() function is used to create a new router object. This function is used when you want to create a new router object in your program to handle requests.

Syntax:

express.Router([options])

Optional Parameters:

- case-sensitive: This enables case sensitivity.
- mergeParams: It preserves the req.params values from the parent router.
- strict: This enables strict routing.

Express express.Router() Function

```
const express = require('express');
const app = express();
const PORT = 3000;
// Single routing
const router = express.Router();
router.get('/', function (req, res, next) {
        console.log("Router Working");
        res.end();
})
app.use(router);
app.listen(PORT, function (err) {
        if (err) console.log(err);
        console.log("Server listening on PORT", PORT);
});
```

Express express.Router() Function

Output:

Console Output:

```
Server listening on PORT 3000
```

Browser Output:

Now open your browser and go to **http://localhost:3000/**, you will see the following output on your screen:

```
Server listening on PORT 3000

Router Working
```

Express express.Router() Function

Example 2:

```
const express = require('express');
const app = express();
const PORT = 3000;

// Multiple routing
const router1 = express.Router();
const router2 = express.Router();
const router3 = express.Router();
```

Express express.Router() Function

```
router1.get('/user', function (req, res, next) {
    console.log("User Router Working");
    res.end();
});

router2.get('/admin', function (req, res, next) {
    console.log("Admin Router Working");
    res.end();
});

router3.get('/student', function (req, res, next) {
    console.log("Student Router Working");
    res.end();
});
```

Express express.Router() Function

```
app.use(router1);
app.use(router2);
app.use(router3);

app.listen(PORT, function (err) {
    if (err) console.log(err);
    console.log("Server listening on PORT", PORT);
});
```

Express express.Router() Function

Output:

```
Server listening on PORT 3000

User Router Working

Admin Router Working

Student Router Working
```

ExpressJS - HTTP Methods

The HTTP method is supplied in the request and specifies the operation that the client has requested. The following table lists the most used HTTP methods –

1 GET
The GET method requests a representation of the specified resource. Requests using GET should only retrieve data and should have no other effect.

2 POST
The POST method requests that the server accept the data enclosed in the request as a new object/entity of the resource identified by the URI.

ExpressJS - HTTP Methods

3 PUT

The PUT method requests that the server accept the data enclosed in the request as a modification to existing object identified by the URI. If it does not exist then the PUT method should create one.

4 DELETE

The DELETE method requests that the server delete the specified resource.

ExpressJS - URL Building

We can now define routes, but those are static or fixed. To use the dynamic routes, we SHOULD provide different types of routes.

Using dynamic routes allows us to pass parameters and process based on them.

ExpressJS - URL Building

Here is an example of a dynamic route –

```
var express = require('express');

var app = express();

app.get('/:id', function(req, res){

   res.send('The id you specified is ' + req.params.id);

});

app.listen(3000);
```

ExpressJS – URL Building

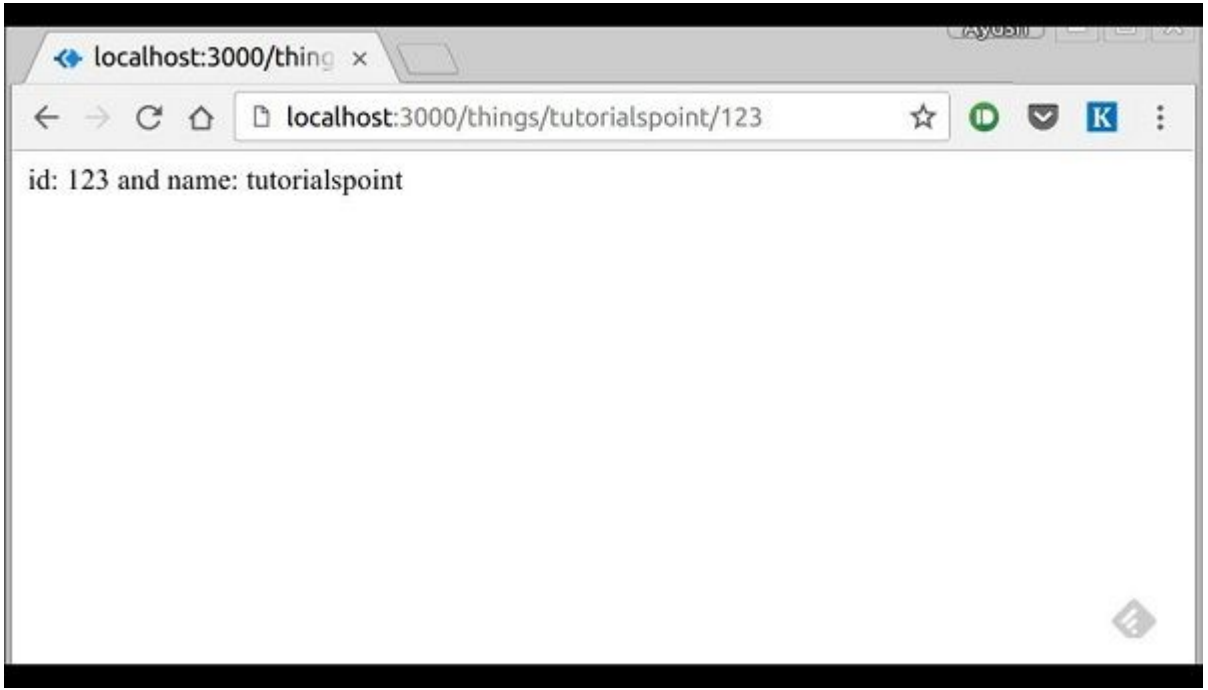
To test this go to **http://localhost:3000/123**. The following response will be displayed.

complex example

```
var express = require('express');

var app = express();


app.get('/things/:name/:id', function(req, res) {

   res.send('id: ' + req.params.id + ' and name: ' +
req.params.name);

});

app.listen(3000);
```

complex example

To test the above code, go to **http://localhost:3000/things/tutorialspoint/12345**.



You can use the **req.params** object to access all the parameters you pass in the url.

For example,

```
var express = require('express');
var app = express();

//Other routes here
app.get('*', function(req, res){
   res.send('Sorry, this is an invalid URL.');
});
app.listen(3000);
```

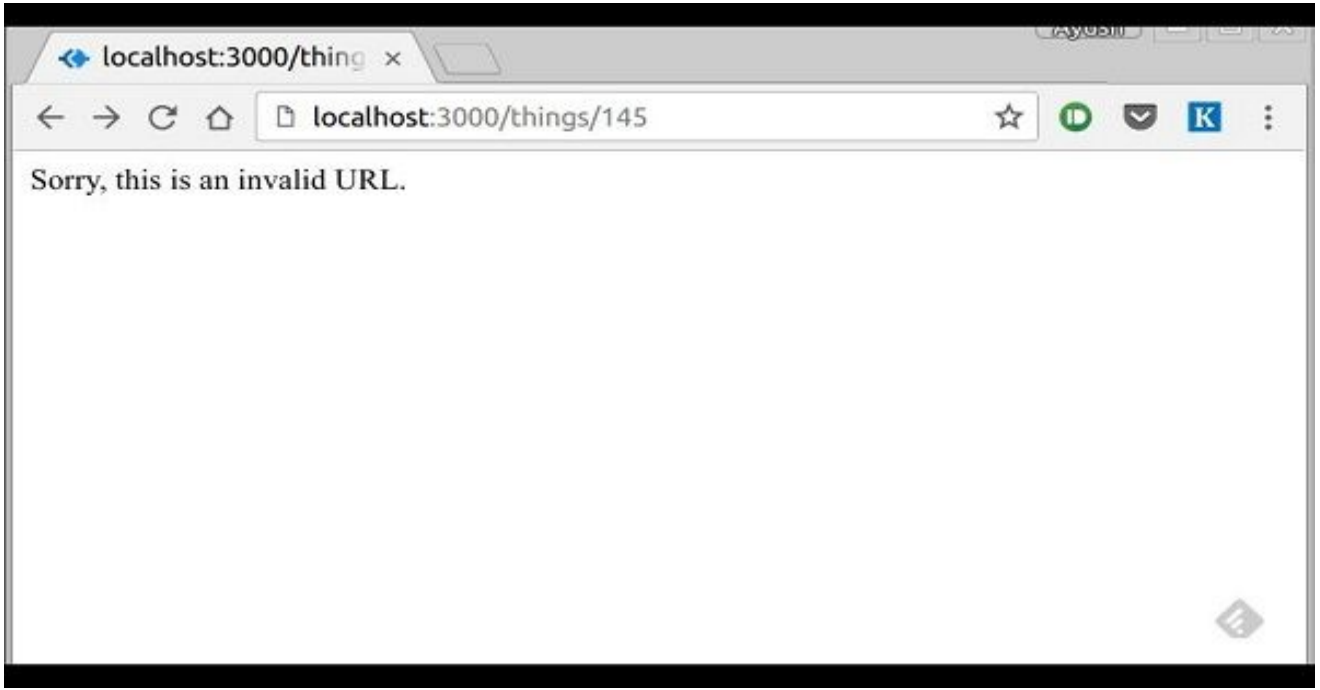
Important – This should be placed after all your routes, as Express matches routes from start to end of the **index.js** file, including the external routers you required.

For example,

if we define the same routes as above, on requesting with a valid URL, the following output is displayed. –

For example,

While for an incorrect URL request, the following output is displayed.



ExpressJS - Middleware

Middleware functions are the functions that access to the request and response object(req, res) in request-response cycle.

A middleware function can perform the following tasks:

- It can execute any code.
- It can make changes to the request and the response objects.
- It can end the request-response cycle.
- It can call the next middleware function in the stack.

ExpressJS - Middleware

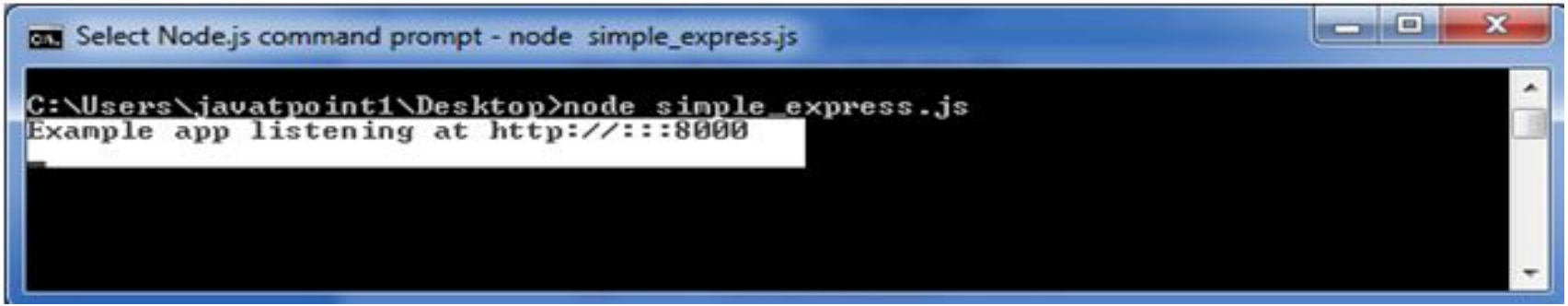
Following is a list of possibly used middleware in Express.js app:

- Application-level middleware
- Router-level middleware
- Error-handling middleware
- Built-in middleware
- Third-party middleware

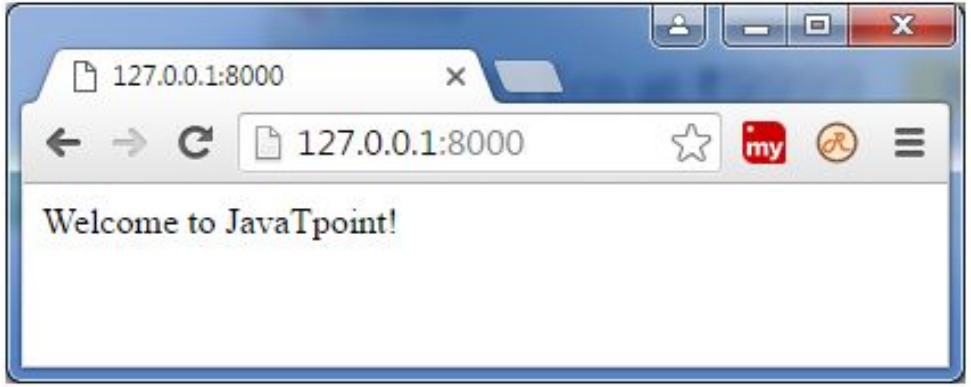
Let's take the most basic Express.js app:

```
var express = require('express');
var app = express();

app.get('/', function(req, res) {
  res.send('Welcome to JavaTpoint!');
});
app.get('/help', function(req, res) {
  res.send('How can I help You?');
});
var server = app.listen(8000, function () {
  var host = server.address().address
  var port = server.address().port
console.log("Example app listening at http://%s:%s", host, port)
})
```

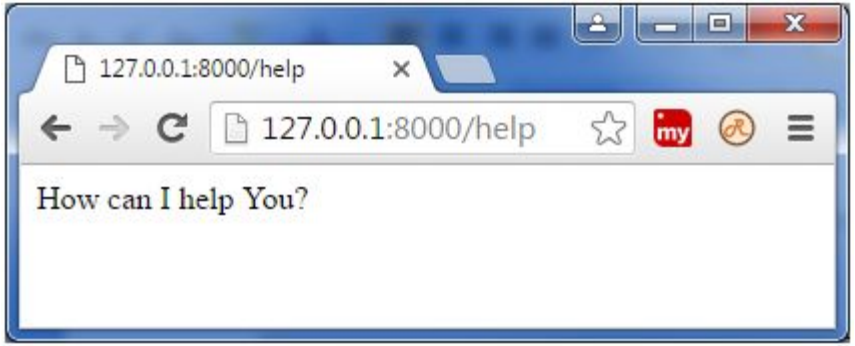
output

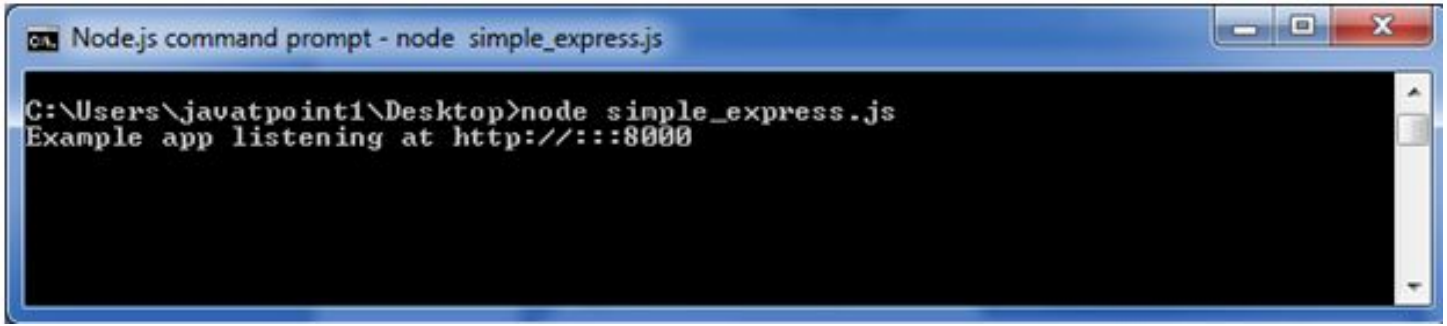


Now, you can see the result generated by server on the local host http://127.0.0.1:8000



Let's see the next page: http://127.0.0.1:8000/help





Use of Express.js Middleware

If you want to record every time you a get a request then you can use a middleware.

File: simple_middleware.js

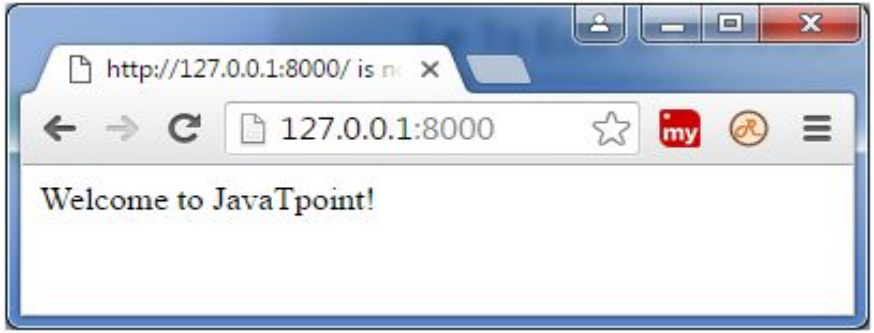
```
var express = require('express');
var app = express();
app.use(function(req, res, next) {
  console.log('%s %s', req.method, req.url);
  next();
});
app.get('/', function(req, res, next) {
  res.send('Welcome to JavaTpoint!');
});
```

```
app.get('/help', function(req, res, next) {
  res.send('How can I help you?');
});
var server = app.listen(8000, function () {
  var host = server.address().address
  var port = server.address().port
  console.log("Example app listening at http://%s:%s", host, port)
})
```

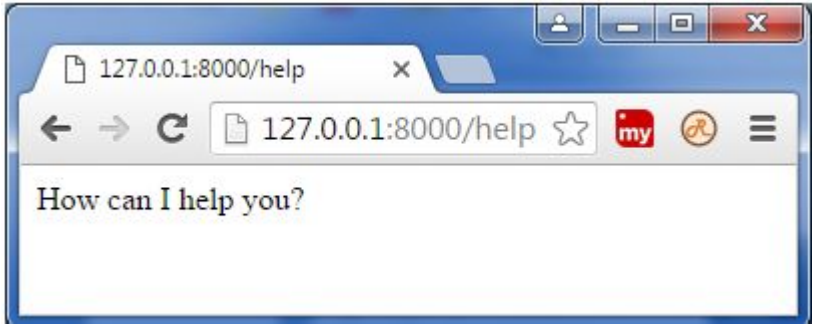
You see that server is listening.

Now, you can see the result generated by server on the local host http://127.0.0.1:8000

Output:



You can see that output is same but command prompt is displaying a GET result.


127.0.0.1:8000/help
127.0.0.1:8000/help
How can I help you?


Node.js command prompt - node simple_middleware.js
C:\Users\javatpoint1\Desktop>node simple_middleware.js
Example app listening at http://:::8000
GET /
GET /help


Node.js command prompt - node simple_middleware.js
C:\Users\javatpoint1\Desktop>node simple_middleware.js
Example app listening at http://:::8000
GET /
GET /help
GET /help

Middleware example explanation

- In the above middleware example a new function is used to invoke with every request via app.use().
- Middleware is a function, just like route handlers and invoked also in the similar manner.
- You can add more middlewares above or below using the same API.

Express.js Cookies Management

What are cookies

Cookies are small piece of information i.e. sent from a website and stored in user's web browser when user browses that website.

Every time the user loads that website back, the browser sends that stored data back to website or server, to recognize user.

Cookies

A cookies generally consist of the following

- Name
- Value
- Zero or more attributes in key-value format. These attributes can include cookies expiration, domain, and flags, etc.

  In order to use cookies in express, first you need to install the cookies-parser middleware through npm into the node modules folder that is already present in your project folder.

  npm install cookies-parser

Import cookie-parser into your app.

```
var express = require('express');
var cookieParser = require('cookie-parser');
var app = express();
app.use(cookieParser());
```

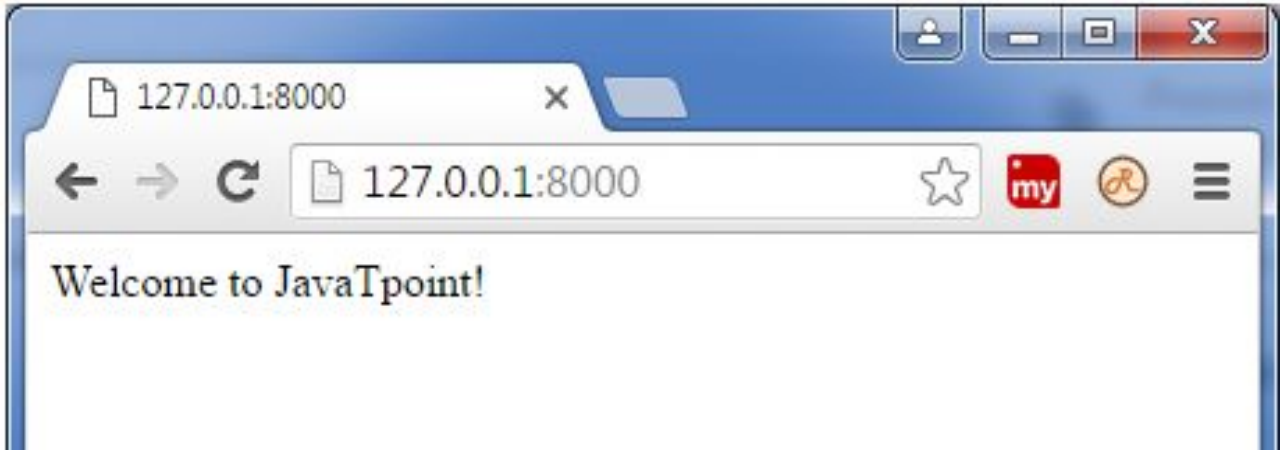
Express.js Cookies Example

File: cookies_example.js

```
var express = require('express');
var cookieParser = require('cookie-parser');
var app = express();
app.use(cookieParser());
app.get('/cookieset',function(req, res){
res.cookie('cookie_name', 'cookie_value');
res.cookie('company', 'javatpoint');
res.cookie('name', 'sonoo');

res.status(200).send('Cookie is set');
});
```

```
app.get('/cookieget', function(req, res) {
  res.status(200).send(req.cookies);
});
app.get('/', function (req, res) {
  res.status(200).send('Welcome to JavaTpoint!');
});
var server = app.listen(8000, function () {
  var host = server.address().address;
  var port = server.address().port;
  console.log('Example app listening at http://%s:%s', host, port);
});
```


Node.js command prompt - node cookies_example.js
C:\Users\javatpoint1\Desktop>node cookies_example.js
Example app listening at http://:::8000

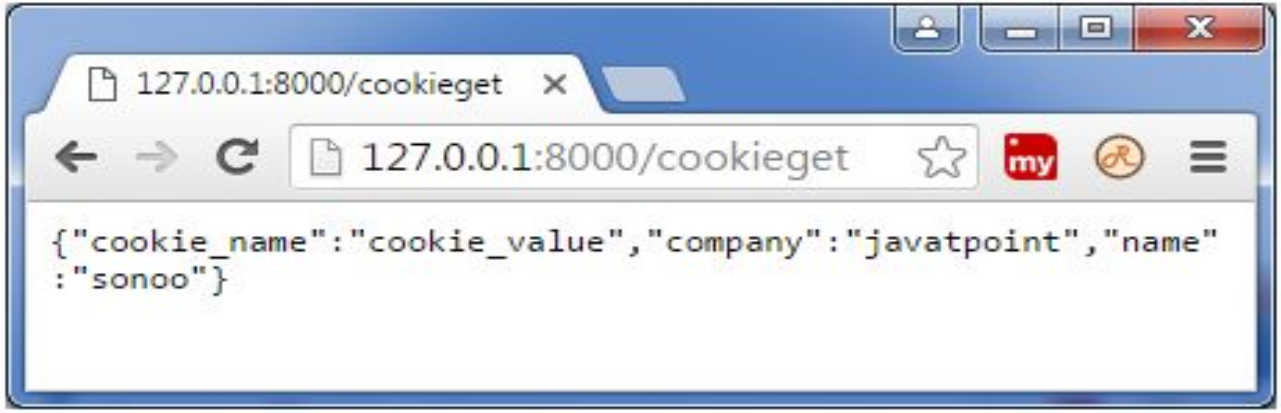

127.0.0.1:8000
127.0.0.1:8000
Welcome to JavaTpoint!

Set cookie:

Now open http://127.0.0.1:8000/cookieset to set the cookie:

Get cookie:

Now open http://127.0.0.1:8000/cookieget to get the cookie:



REST API

REST is an acronym for REpresentational State Transfer and an architectural style for distributed hypermedia systems.

Roy Fielding first presented it in 2000 in his famous dissertation.

Since then it has become one of the most widely used approaches for building web-based APIs (*Application Programming Interfaces*).

REST API mostly use JSON as the preferred choice for transferring data as they are easy to understand and is readable.

REST API

REST is set of methods in which HTTP clients can request information from server via the HTTP protocol.

It is an application program interface (API) which makes use of HTTP request to GET, PUT, POST and DELETE the data over WWW.

REST API

The main functions used in any REST based architecture are:

1. GET : GET means to read the data. The function of this operation is to retrieve the data from the database and present it to the User.
2. POST: POST , is used to POST/ADD new data to the database. It allows users to add new data to the database.
3. PUT: PUT means update the data already present in the database.
4. DELETE : It is used to delete any existing data from the database.

Six Guiding Principles of REST

Uniform Interface

By applying the principle of generality to the components interface, we can simplify the overall system architecture and improve the visibility of interactions.

The following four constraints can achieve a uniform REST interface:

- Identification of resources
- Manipulation of resources through representations
- Self-descriptive messages
- Hypermedia as the engine of application state

Six Guiding Principles of REST

Client-Server

The client-server design pattern enforces the separation of concerns, which helps the client and the server components evolve independently.

By separating the user interface concerns (client) from the data storage concerns (server), we improve the portability of the user interface across multiple platforms and improve scalability by simplifying the server components.

Six Guiding Principles of REST

Stateless

Statelessness mandates that each request from the client to the server must contain all of the information necessary to understand and complete the request.

The server cannot take advantage of any previously stored context information on the server.

For this reason, the client application must entirely keep the session state.

Six Guiding Principles of REST

Cacheable

The cacheable constraint requires that a response should implicitly or explicitly label itself as cacheable or non-cacheable.

If the response is cacheable, the client application gets the right to reuse the response data later for equivalent requests and a specified period.

Six Guiding Principles of REST

Layered System

The layered system style allows an architecture to be composed of hierarchical layers by constraining component behavior. In a layered system, each component cannot see beyond the immediate layer they are interacting with.

A layman's example of a layered system is the *MVC pattern*. The MVC pattern allows for a clear separation of concerns, making it easier to develop, maintain, and scale the application.

Six Guiding Principles of REST

Code on Demand

REST also allows client functionality to extend by downloading and executing code in the form of applets or scripts.

The downloaded code simplifies clients by reducing the number of features required to be pre-implemented. Servers can provide part of features delivered to the client in the form of code, and the client only needs to execute the code.

Express Generator

Express generator is an application generator tool which is use to quickly create an application skeleton

Like express JS, express Generator is also a Node js Module.. It is used to quick start and develop express js applications very easily. It does not come as a part of Node Js platform basic installation.

Express generator is a package that simply saves your time and effort in creating node js web apps. It does this by generating folders that you would otherwise have created manually.

Express Generator

To install express Js globally

Open command prompt and execute this command

npm install -g express generator

After express generator installation is done , we need to check whether this module is installed successfully or not. If you see “ express-generator folder under “node_ modules” folder. That means express generator is installed successfully.

Express Generator

Folders we have created are as follows

Bin contains the port which our app has ti listen to

Node_modules : this folder contains numerous packages and functionality necessary for our app to function.

Public: this folder holds all the images, third party javascript files & stylesheets

Routes (contains index and user files by default) : the routes folder determines links that will be functional.

Express Generator

Views: the views allow us to create HTML templates that can be applied to pages

app.js : this is the entry point to our app. In the app.js the dependencies are required. The routes are also required and we tell express to use the routes. It also contains other configurations to tell express how to handle errors.

Package.json: this file contains the list of dependencies

Package-lock.json: this folder contains details about the application

Session Management using express-session Module in Node.js

Session management can be done in node.js by using the express-session module. It helps in saving the data in the key-value form.

Installation of express-session module:

1. You can visit the link Install express-session module. You can install this package by using this command.

   ```
   npm install express-session
   ```

2. After installing express-session you can check your express-session version in command prompt using the command.

   ```
   npm version express-session
   ```

3. After that, you can create a folder and add a file for example index.js, To run this file you need to run the following command.

   ```
   node index.js
   ```

```
const express = require("express")
const session = require('express-session')
const app = express()

// Port Number Setup
var PORT = process.env.port || 3000

// Session Setup
app.use(session({

    // It holds the secret key for session
    secret: 'Your_Secret_Key',

    // Forces the session to be saved
    // back to the session store
    resave: true,

    // Forces a session that is "uninitialized"
    // to be saved to the store
    saveUninitialized: true
}))
```

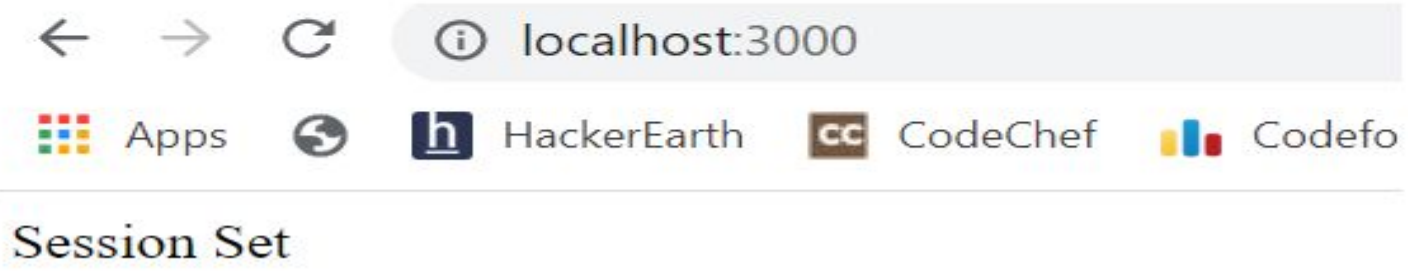
. Run index.js file using below command:

```
node index.js
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL                                    2: node
PS C:\Users\Lenovo\Downloads\GFG Internship\Express Session Module Demo> node index.js
Server created Successfully on PORT : 3000
```

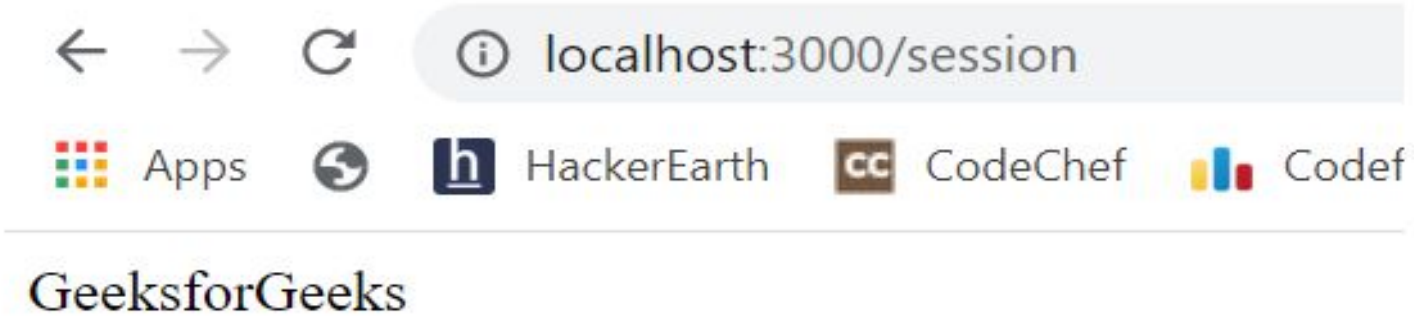
Now to set your session, just open browser and type this URL:

```
http://localhost:3000/
```



Till now, you have set session and to see session value, type this URL:

```
http://localhost:3000/session
```



So this is how you can do session management in node.js using the express-session module

```
app.get("/", function(req, res){

    // req.session.key = value
    req.session.name = 'GeeksforGeeks'
    return res.send("Session Set")
})

app.get("/session", function(req, res){

    var name = req.session.name
    return res.send(name)

    /*  To destroy session you can use
        this function
     req.session.destroy(function(error){
        console.log("Session Destroyed")
    })
    */
})

app.listen(PORT, function(error){
    if(error) throw error
    console.log("Server created Successfully on PORT :", PORT)
})
```

Difference between EXPRESS.JS & NODE.JS

Node.Js	Express.Js
It is used for building both the frontend and backend of web application	It is a node.js framework that is used for building the backend of a web application
It was developed on google's V8 Javascript engine as a cross platform runtime environment	It is a framework for node.js
It was written using different programming languages like JavaScript, C, C++	It was written using JavaScript
It is not a web framework	It is a web framework

Difference between EXPRESS.JS & NODE.JS

Node.Js	Express.Js
Developers need to install NodeJs on their computer system to use it.	Developers need to install ExpressJs along with NodeJs to use it
Used for developing server side and networking applications.	used for building server side application on Node Js
It can be used both on the server side and client side	It can be used on the server side
It provides many features for developers to build a web application.	It provides routing components and supports middleware to make web app development easier
It provide many features to programmer to write database queries	Programmers can write database queries on expressjs because it is a framework of nodeJs.

Difference between EXPRESS.JS & NODE.JS

Node.Js	Express.Js
It supports other languages like typescript, coffeescript and ruby	It supports JavaScript
It is used by PayPal, Walmart, LinkedIn, Uber , etc	It is used by PayPal, IBM, Fox Sports, etc.